

Complemento Práctico: Diseño de Jerarquías Complejas

por igual. Pero, ¿qué pasa cuando necesitamos recuperar la identidad específica de un objeto? En esta actividad, aprenderemos a realizar Downcasting de forma segura, entendiendo la diferencia entre "convertir" y "desenmascarar".

Luego, entraremos en el terreno del Diseño Abstracto. Analizaremos por qué algunas clases, como "Figura Geométrica" o "Carta", están destinadas a no existir nunca como objetos concretos, sino como moldes (contratos) para sus hijos. Finalizaremos desglosando el ejercicio de la Baraja Inglesa, un caso real donde la herencia y el polimorfismo nos ahorran escribir cientos de líneas de código mediante algoritmos de generación automática.

Contenido



Downcasting: Bajando por la Jerarquía

Aprendizaje sobre downcasting seguro y el uso de instanceof.

Clases Abstractas: Conceptos que No Nacen

Exploración de clases abstractas y por qué no pueden ser instanciadas.

Métodos Abstractos: El Contrato Obligatorio

Análisis de métodos abstractos como contratos para las clases hijas.

Caso Baraja: Diseñando la Jerarquía de Cartas

Estudio de caso sobre el diseño de la jerarquía de una baraja de cartas.

Generación Automática y Polimorfismo

Aplicación de polimorfismo para la generación automática de elementos.

Downcasting: Bajando por la Jerarquía

El Upcasting (Hijo → Padre) es automático y seguro. Es como guardar una Guitarra en una caja etiquetada "Instrumento". El Downcasting (Padre → Hijo) es manual y peligroso. Es decirle al compilador: "Confía en mí, yo sé que lo que hay en esta caja de Instrumento es realmente una Guitarra".

El compilador confía en ti, pero si te equivocas, el programa explota con un **ClassCastException**. Si en la caja había un Piano e intentas forzarlo a ser Guitarra, ocurrirá un error en tiempo de ejecución. Por eso necesitamos una estrategia de seguridad antes de realizar cualquier downcasting.

El Riesgo Mortal y su Solución

El Problema

Hacer downcasting sin verificar es como saltar sin red de seguridad. El compilador acepta el código, pero en tiempo de ejecución puede fallar catastróficamente.

La Solución: instanceof

Siempre debemos preguntar antes de "desenmascarar". El operador instanceof nos permite verificar el tipo real del objeto antes de hacer la conversión.

```
// Escenario: Lista heterogénea (mezcla de instrumentos)
for (Instrumento ins : orquesta) {
    // PREGUNTA DE SEGURIDAD
    if (ins instanceof Guitarra) {
        // DOWNCASTING SEGURO
        Guitarra g = (Guitarra) ins;
        g.tocarSolo(); // Método exclusivo de Guitarra
    }
}
```

Este patrón se usa constantemente en aplicaciones profesionales. Por ejemplo, cuando procesamos eventos de usuario, archivos de diferentes formatos o cualquier colección polimórfica donde necesitamos acceder a funcionalidades específicas de ciertos tipos.

Clases Abstractas: Conceptos que No Nacen

En el video introducimos la palabra clave `abstract`. Hay conceptos que son demasiado vagos para ser instanciados. ¿Existe un "Animal" que no sea ni perro, ni gato, ni pez? No. "Animal" es una categoría mental, un concepto organizador que nos ayuda a pensar, pero que nunca se materializa por sí mismo.

La Regla de Oro: Si una clase es abstracta, está prohibido usar `new` con ella.

```
Carta c = new Carta(); // ERROR: Carta is abstract
```

¿Por qué prohibir el `new`? Para obligar a cumplir un contrato. Al hacer la clase abstracta, podemos crear métodos abstractos (sin cuerpo). Esto funciona como una ley marcial para los hijos: "Si quieres heredar de mí, ESTÁS OBLIGADO a implementar este método". Esta obligación garantiza que todos los descendientes proporcionen su propia implementación específica.

Métodos Abstractos: El Contrato Obligatorio

El Poder del Contrato

Los métodos abstractos son declaraciones sin implementación. Son promesas que la clase padre exige a sus hijos. Cada hijo debe decidir *cómo* cumplir ese contrato, pero no puede ignorarlo.

```
public abstract class Carta {  
    // Obligo a mis hijos a que  
    // ellos decidan cómo mostrarse  
    public abstract String  
        getRepresentacion();  
}
```

Este mecanismo es fundamental para el polimorfismo: garantizamos que todos los hijos responden al mismo mensaje, pero cada uno a su manera.

Caso Baraja: Diseñando la Jerarquía de Cartas

El ejercicio pide modelar una baraja. Analicemos las decisiones de diseño del video. Toda buena arquitectura comienza identificando qué tienen en común los objetos y qué los diferencia. En una baraja, todas las cartas comparten ciertas características: tienen un palo (corazones, diamantes, tréboles, picas) y un estado (tapada o destapada). Pero su representación varía significativamente.

1

La Clase Base (Carta)

Todas tienen Palo y estado (tapada). Como no existe una "Carta genérica" (siempre es un 4, o una J, etc.), declaramos a Carta como `ABSTRACTA`.

2

División en Subclases

Notamos dos comportamientos distintos: Cartas Numéricas (con valor `int`: 2, 3... 10) y Cartas Figura (con letra `String`: J, Q, K, A).

Esta división no es arbitraria: surge de analizar cómo se representan realmente las cartas. Los números y las figuras tienen naturalezas diferentes, y merecen clases separadas que capturen esa diferencia.

Implementación de Cartas Numéricas

Características Únicas

Las cartas numéricas tienen un atributo exclusivo: su número (del 2 al 10). Este valor determina completamente su identidad junto con su palo.

- Almacenan un valor entero
- Heredan el palo de la clase base
- Implementan su propia representación

```
public class CartaNumerica
    extends Carta {

    private int numero;
    // Atributo exclusivo

    public CartaNumerica(
        int numero,
        String palo) {

        super(palo);
        // Llama al constructor
        // de la clase abstracta

        this.numero = numero;
    }

    @Override
    public String
        getRepresentacion() {
        return "Soy el " + numero;
    }
}
```

Observa cómo el constructor usa `super(palo)` para inicializar la parte heredada, y luego maneja su atributo específico. El método `getRepresentacion()` cumple el contrato abstracto de la clase padre.

Generación Automática: El Poder del Polimorfismo

El punto más fuerte del video es cómo llenar el mazo. En lugar de escribir 52 líneas individuales (new Carta...), usamos bucles inteligentes que aprovechan el polimorfismo. Este es el verdadero poder de la programación orientada a objetos: automatizar la creación de familias de objetos relacionados.



Desafío Matemático

Los índices van del 1 al 13, pero las cartas numéricas solo llegan al 10. Las 11, 12 y 13 son figuras.



Lógica Condicional

Usamos un condicional dentro del bucle para decidir qué tipo de objeto crear en cada iteración.



Lista Polimórfica

La lista cree que tiene "Cartas" genéricas, sin saber que contiene una mezcla de hijos diferentes.

```
ArrayList<Carta> mazo = new ArrayList<>();  
// Lista Polimórfica
```

```
for (String palo : palos) {  
    for (int i = 1; i <= 13; i++) {  
        if (i <= 10) {  
            // CAMINO A: Crear Carta Numérica  
            mazo.add(new CartaNumerica(i, palo));  
        } else {  
            // CAMINO B: Crear Figura  
            String letra = convertirNumeroALetra(i);  
            mazo.add(new CartaFigura(letra, palo));  
        }  
    }  
}
```

Ventajas de la Arquitectura Polimórfica

Resultado: Código Elegante y Mantenible

La lista `mazo` cree que tiene "Cartas". No sabe que adentro hay una mezcla de hijos. Esto permite barajar y repartir sin importar el tipo específico. Cualquier operación que definamos sobre "Carta" funcionará automáticamente con `CartaNumerica` y `CartaFigura`.

Si mañana necesitamos agregar un Joker, simplemente creamos una nueva clase que herede de `Carta`, e insertamos una línea en el algoritmo de generación. No necesitamos modificar el código de barajar, repartir, o mostrar cartas. **Esto es extensibilidad real.**

Resumen: Hacia la Programación Profesional

Hemos avanzado hacia la programación profesional integrando tres pilares fundamentales del diseño orientado a objetos. Cada uno de estos conceptos no es solo teoría académica: son herramientas que usarás diariamente en proyectos reales.

Seguridad con Downcasting

Sabemos que el Downcasting es útil para recuperar funcionalidades específicas, pero requiere chequeo previo con `instanceof`. Nunca confíes ciegamente en un cast.

Abstracción como Contrato

Aprendimos a usar `abstract` no solo para prohibir objetos, sino para definir contratos (métodos abstractos) que estandarizan el comportamiento de los hijos.

Diseño Polimórfico

En el ejercicio de la Baraja, vimos la potencia real del polimorfismo: una lista de tipo `Padre` puede almacenar múltiples tipos de `Hijos`, permitiéndonos manejar todo el mazo con un solo ciclo genérico.

"El buen diseño no se trata de escribir menos código, sino de escribir código que sea fácil de entender, extender y mantener. Las jerarquías abstractas bien diseñadas son la clave para lograrlo."